Object Oriented Programming

Mihai Dascălu

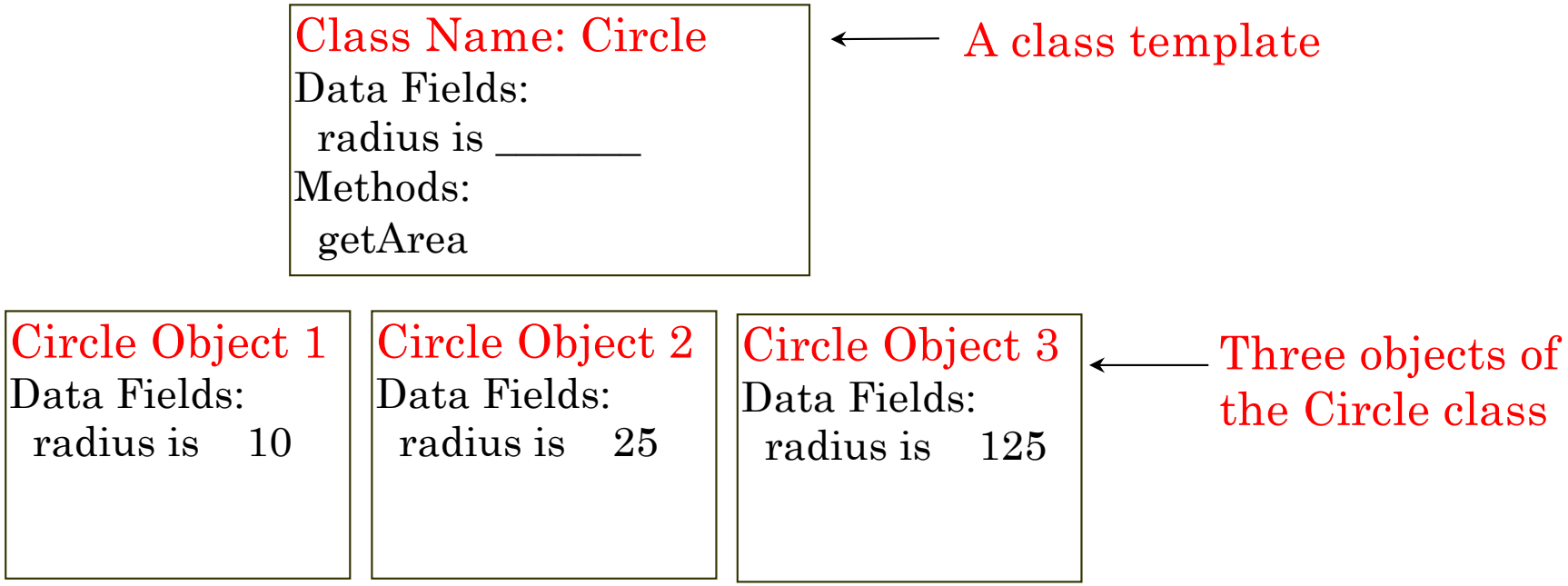
Thinking in Objects

Objects, more Objects and Classes

OO Programming Concepts

- Object-oriented programming (OOP) involves programming using objects
- *Object* = an entity in the real world that can be distinctly identified
- An object has a unique identity, state, and behaviors
- The *state* of an object consists of a set of *data fields* (also known as *properties*) with their current values
- The *behavior* of an object is defined by a set of methods
- An object wraps the data (variables) and code acting on the data (methods) together as a single unit

Objects



- An object has both a state and a behavior
- The state defines the object, whereas the behavior defines what the object does

Classes

- *Classes* are constructs that define objects of the same type
- A Java class uses
  - *Variables* to define data fields
  - *Methods* to define behaviors
- Additionally, a class provides a special type of methods, known as **constructors**, which are invoked to construct (build) objects from the class

Classes

```
class Circle {
  /** The radius of this circle */
  double radius = 1.0;
  /** Construct a circle object */
  Circle() {
  }
  Circle(double newRadius) {
    radius = newRadius;
  }
  /** Return the area of this circle */
  double getArea() {
    return radius * radius * 3.14159;
  }
}
```

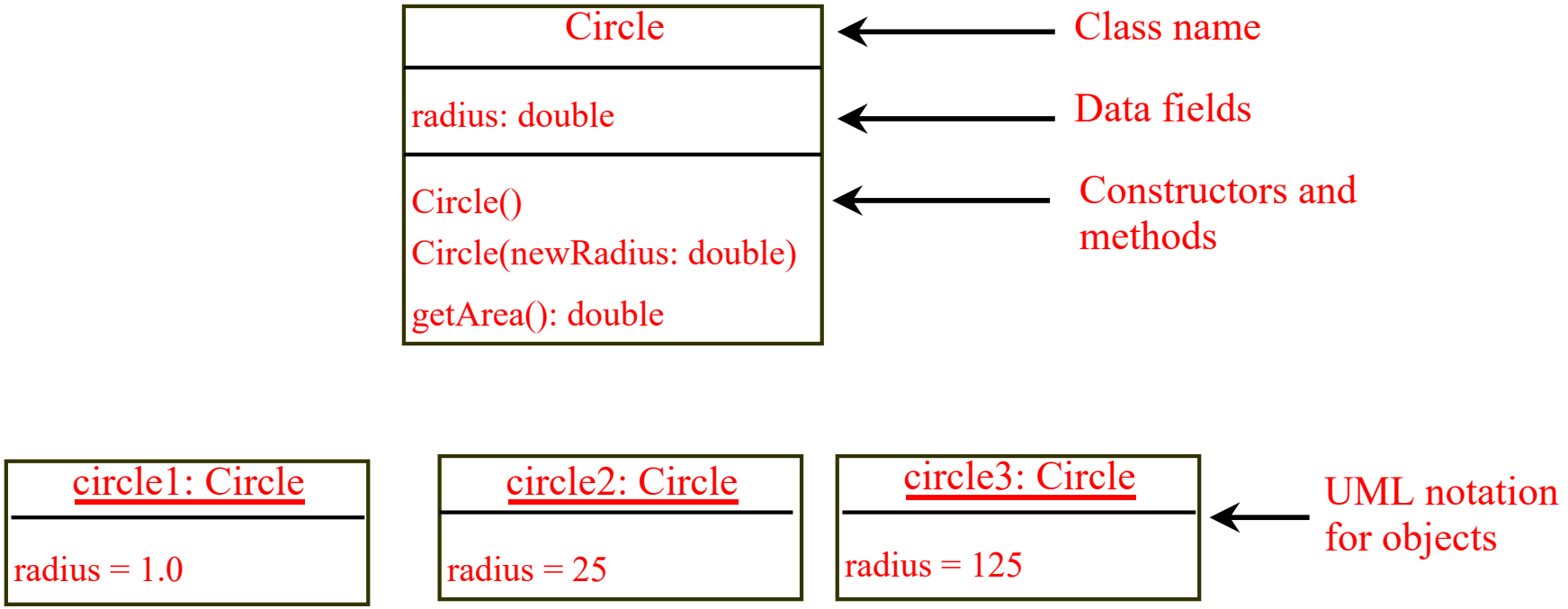
Data field

Constructors

Method

Circle

UML Class Diagram



Constructors

- Constructors are a special kind of methods that are invoked to build objects
- A constructor with no parameters is referred to as a *no-arg constructor*
  - Constructors must have the same name as the class itself
  - Constructors **do not have a return type**, not even void
  - Constructors are invoked using the **new** operator when an object is created
  - Constructors play the role of initializing objects

Creating Objects Using Constructors

```
new ClassName();
```

Example:

```
new Circle();

new Circle(5.0);
```

```
Circle() {
}

Circle(double newRadius) {
  radius = newRadius;
}
```

Default Constructor

- A class may be declared without constructors
- In this case, a no-arg constructor with an empty body is *implicitly* declared in the class =>
  - default constructor provided automatically **only if no constructors** are explicitly declared in the class

Declaring Object Reference Variables

- To reference an object, assign the object to a reference variable
- To declare a reference variable, use the syntax:

```
ClassName objectRefVar;
```

e.g.

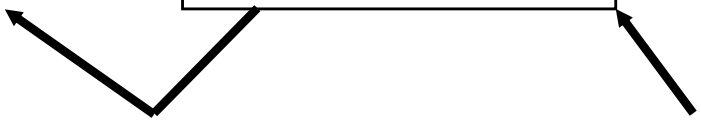
```
Circle myCircle;
```

Declaring/Creating Objects in a Single Step

```
ClassName objectRefVar = new ClassName();
```

Example:

```
Circle myCircle = new Circle();
```



Assign object reference Create an object

Accessing Objects

- Referencing the object's data:

```
objectRefVar.data
```

e.g., `myCircle.radius`

- Invoking the object's method:

```
objectRefVar.methodName(arguments)
```

e.g., `myCircle.getArea()`

Caution

- Recall that you use

  Math.methodName(arguments) (e.g., Math.pow(3, 2.5))

to invoke a method in the Math class

- Can you invoke getArea() using Circle1.getArea()? **No**
- All the methods defined beforehand are static, which are defined using the static keyword
- However, getArea() is non-static and it must be invoked from an object using

  objectRefVar.methodName(arguments)

  (e.g., myCircle.getArea())

Reference Data Fields

- The data fields can be of reference types
  - e.g., the following Student class contains a data field name of the String type

```
public class Student {
  String name; // default value null
  int age; // default value 0
  boolean isScienceMajor; // default value false
  char gender; // default value '\u0000'
}
```

The null Value

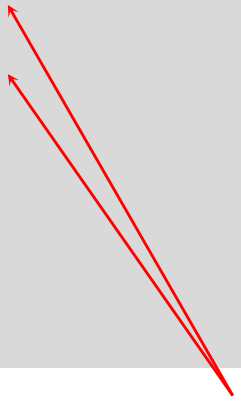
- If a data field of a reference type does not reference any object, the data field holds a special literal value = **null**
- The default value of a data field:
  - null for a reference type
  - 0 for a numeric type, false for a boolean type
  - '\u0000' for a char type

```
public class Test {
  public static void main(String[] args) {
    Student s = new Student();
    System.out.println("name? " + s.name);
    System.out.println("age? " + s.age);
    System.out.println("isScienceMajor? " + s.isScienceMajor);
    System.out.println("gender? " + s.gender);
  }
}
```

Variable initialization

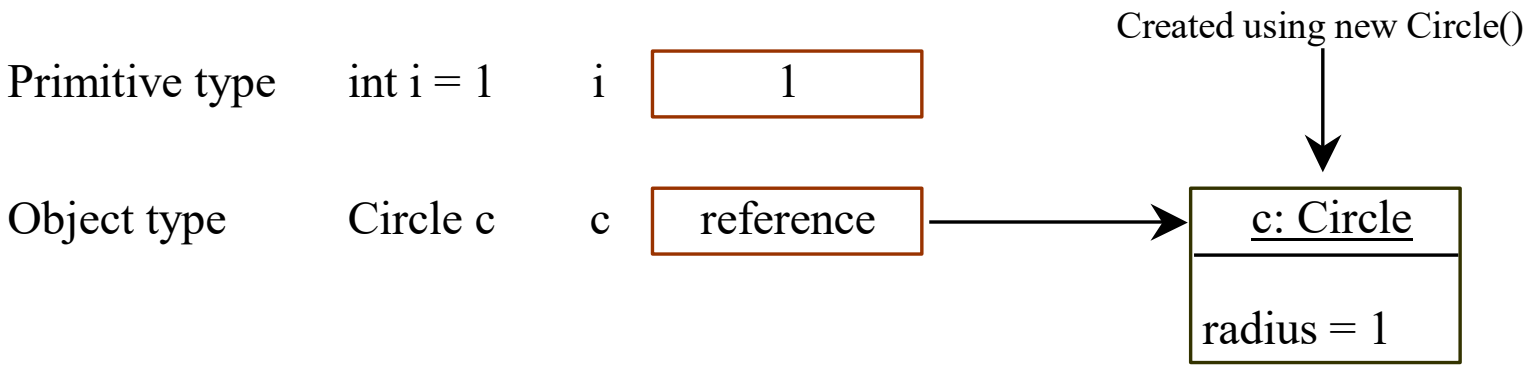
- Java assigns no default value to a local variable inside a method

```
public class Test {
  public static void main(String[] args) {
    int x; // x has no default value
    String y; // y has no default value
    System.out.println("x is " + x);
    System.out.println("y is " + y);
  }
}
```



Compilation Error
variables not initialized

Differences between Variables of Primitive Data Types and Object Types

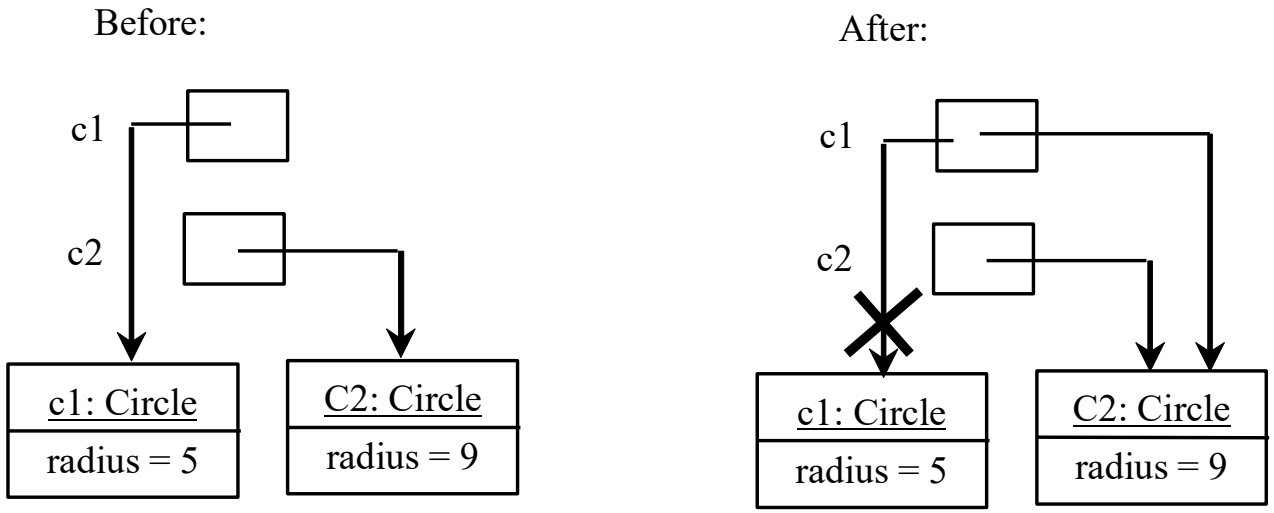


Copying Variables of Primitive Data Types and Object Types

Primitive type assignment i = j



Object type assignment c1 = c2



Garbage Collection (1)

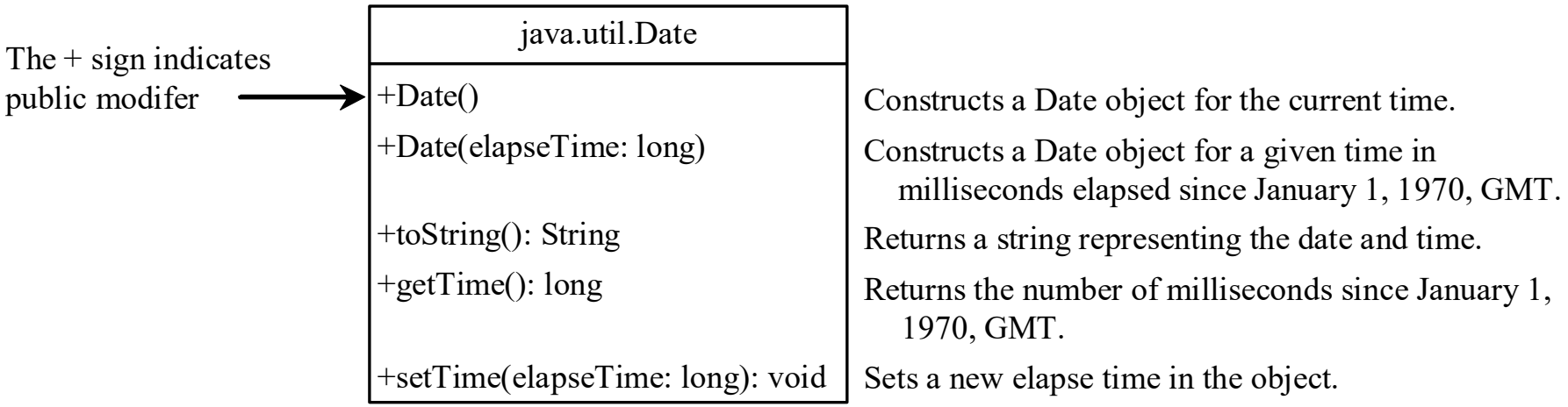
- After c1=c2, c1 points to the same object referenced by c2
  => the previously referenced object is no longer used
  => **garbage**
- Garbage is automatically collected by JVM
- TIP: If you know that an object is no longer needed, you can *explicitly assign null* to a reference variable for the object (or invoke clear() on Collections)
- The JVM will automatically collect the space if the object is not referenced by any variable

Garbage Collection (2)

- System.gc() – best effort to reclaim space from discarded objects
- *Not recommended* – unnecessary performance issues, although there may be cases in which its opportunistic use increases performance
- Should not be used in production (potentially indicates memory leaks)
- Does not stop throwing OutOfMemoryError

The Date Class

- Java provides a system-independent encapsulation of date and time – java.util.Date class
- Use the Date class to create an instance for the current date and time



The Date Class Example

For example,

```
java.util.Date date = new java.util.Date();

System.out.println(date.toString());
```

Displays a string like

```
DayOfWeek Month Day hh:mm:ss EST YEAR
```

PrintCalendar

The Random Class

- Math.random() -> obtain a random double value between 0.0 and 1.0 (excluding 1.0)
- java.util.Random -> a more useful random number generator

java.util.Random	
+Random()	Constructs a Random object with the current time as its seed.
+Random(seed: long)	Constructs a Random object with a specified seed.
+nextInt(): int	Returns a random int value.
+nextInt(n: int): int	Returns a random int value between 0 and n (exclusive).
+nextLong(): long	Returns a random long value.
+nextDouble(): double	Returns a random double value between 0.0 and 1.0 (exclusive).
+nextFloat(): float	Returns a random float value between 0.0F and 1.0F (exclusive).
+nextBoolean(): boolean	Returns a random boolean value.

The Random Class Example

- If two Random objects have the same seed, they will generate identical sequences of numbers.
  - For example

```
Random random1 = new Random(3);
System.out.print("From random1: ");
for (int i = 0; i < 10; i++)
  System.out.print(random1.nextInt(1000) + " ");
Random random2 = new Random(3);
System.out.print("\nFrom random2: ");
for (int i = 0; i < 10; i++)
  System.out.print(random2.nextInt(1000) + " ");
```

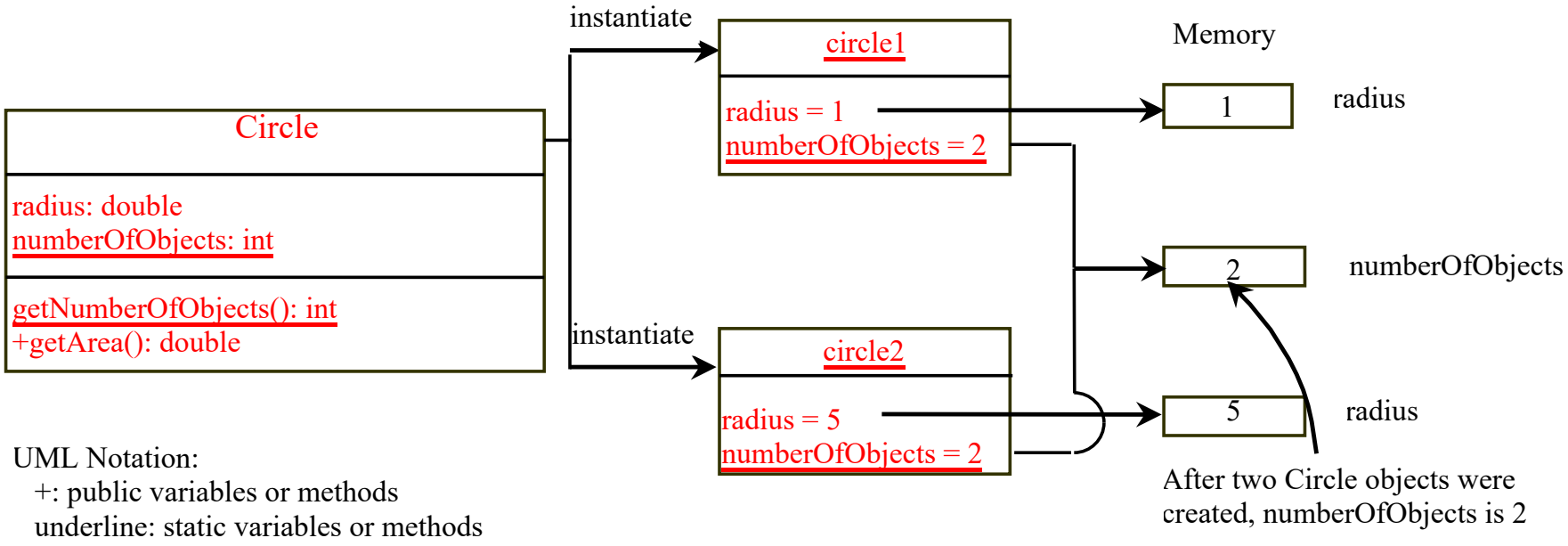
```
From random1: 734 660 210 581 128 202 549 564 459 961
From random2: 734 660 210 581 128 202 549 564 459 961
```

Instance Variables, and Methods

- *Instance variables* belong to a specific instance
- *Instance methods* are invoked by an instance of the class
- *Static variables* are shared by all the instances of the class
- *Static methods* are not tied to a specific object
- *Static constants* are final variables shared by all the instances of the class

Static Variables, Constants, and Methods

- To declare static variables, constants, and methods, use the static modifier.



Animal

Visibility Modifiers and Accessor/Mutator Methods

- By *default*, the class, variable, or method can be accessed by any class in the *same* package
- `public`

  The class, data, or method is visible to any class in any package
- `private`

  The data or methods can be accessed only by the declaring class
- Get and set methods (getters & setters) are used to read and modify private properties

Visibility

package p1;

```
public class C1 {
  public int x;
  int y;
  private int z;

  public void m1() {
  }
  void m2() {
  }
  private void m3() {
  }
}
```

```
public class C2 {
  void aMethod() {
    C1 o = new C1();
    can access o.x;
    can access o.y;
    cannot access o.z;

    can invoke o.m1();
    can invoke o.m2();
    cannot invoke o.m3();
  }
}
```

package p2;

```
public class C3 {
  void aMethod() {
    C1 o = new C1();
    can access o.x;
    cannot access o.y;
    cannot access o.z;

    can invoke o.m1();
    cannot invoke o.m2();
    cannot invoke o.m3();
  }
}
```

package p1;

```
class C1 {
  ...
}
```

```
public class C2 {
  can access C1
}
```

package p2;

```
public class C3 {
  cannot access C1;
  can access C2;
}
```

- *Private* restricts access **within** a **class**
- *Default* restricts access **within** a **package**
- *Public* modifier enables **unrestricted access**

NOTE

- An object cannot access its private members, as shown in (b). It is OK, however, if the object is declared in its own class, as shown in (a).

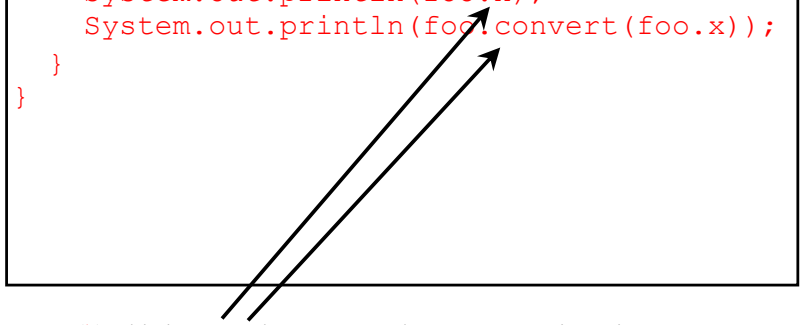
```
public class Foo {
  private boolean x;

  public static void main(String[] args) {
    Foo foo = new Foo();
    System.out.println(foo.x);
    System.out.println(foo.convert());
  }

  private int convert(boolean b) {
    return x ? 1 : -1;
  }
}
```

(a) This is OK because object foo is used inside the Foo class

```
public class Test {
  public static void main(String[] args) {
    Foo foo = new Foo();
    System.out.println(foo.x);
    System.out.println(foo.convert(foo.x));
  }
}
```



(b) This is wrong because x and convert are private in Foo.

Class Encapsulation

- Wrap up data under a single unit
- Variables of a class are hidden from other classes and can be accessed only through the methods of their current class
  - Declare the variables of a class as private
  - Provide public setter and getter methods to modify and view the variable values

Why Encapsulation

- Data Hiding → protection
- Increased flexibility
- Facilitate easy maintenance & Testing code is easy
- Reusability

Circle	
-radius: double	The radius of this circle (default: 1.0).
-numberOfObjects: int	The number of circle objects created.
+Circle()	Constructs a default circle object.
+Circle(radius: double)	Constructs a circle object with the specified radius.
+getRadius(): double	Returns the radius of this circle.
+setRadius(radius: double): void	Sets a new radius for this circle.
+getNumberOfObject(): int	Returns the number of circle objects created.
+getArea(): double	Returns the area of this circle.

The **-** **sign** indicates private modifier

Passing Objects to Methods

- Passing by value for primitive type value
  - The value is passed to the parameter
- Passing by value for reference type value
  - The value is the reference to the object

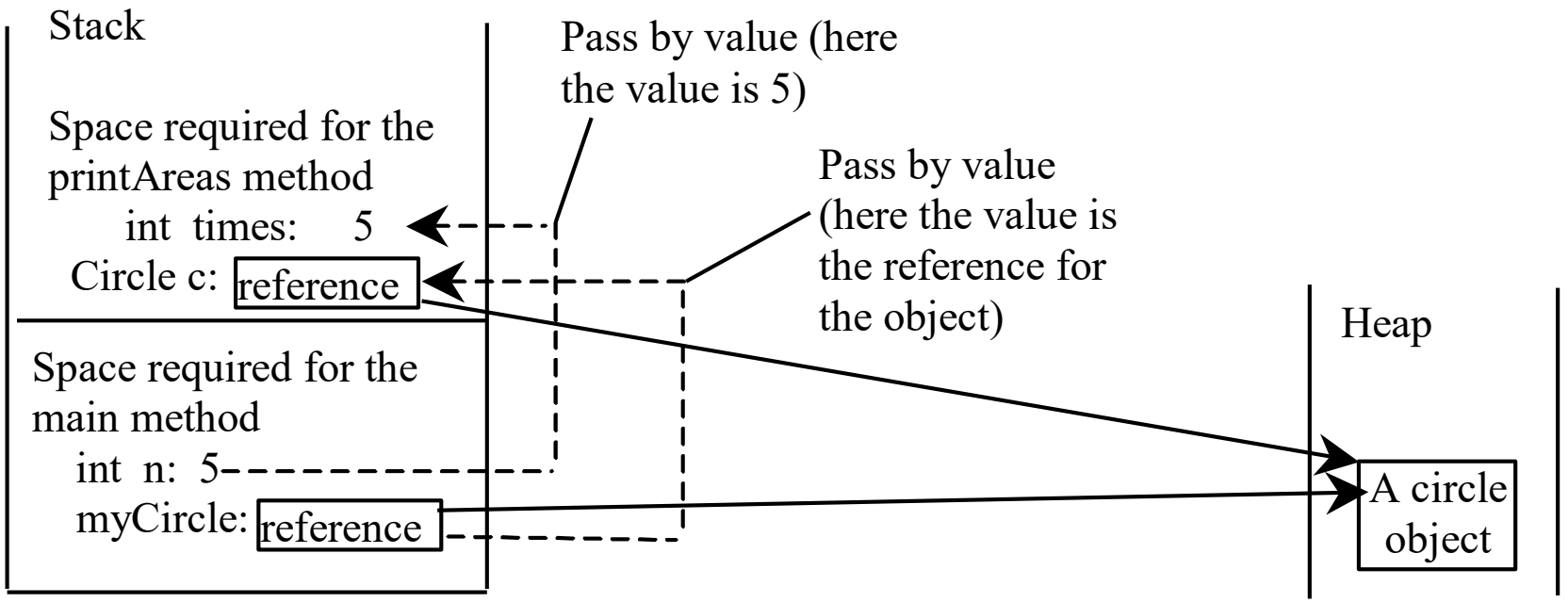
SeriaCB

Passing Objects to Methods

```
public class TestPassObject {
  /** Main method */
  public static void main(String[] args) {
    Circle3 myCircle = new Circle3(1);
    int n = 5;
    printAreas(myCircle, n);
    System.out.println("R = " + myCircle.getRadius());
    System.out.println("n = " + n);
  }

  /** Print a table of areas for radius */
  public static void printAreas(Circle3 c, int times) {
    System.out.println("Radius\tArea");
    while (times >= 1) {
      System.out.println(c.getRadius() + "\t" + c.getArea());
      c.setRadius(c.getRadius() + 1);
      times--;
    }
  }}
```

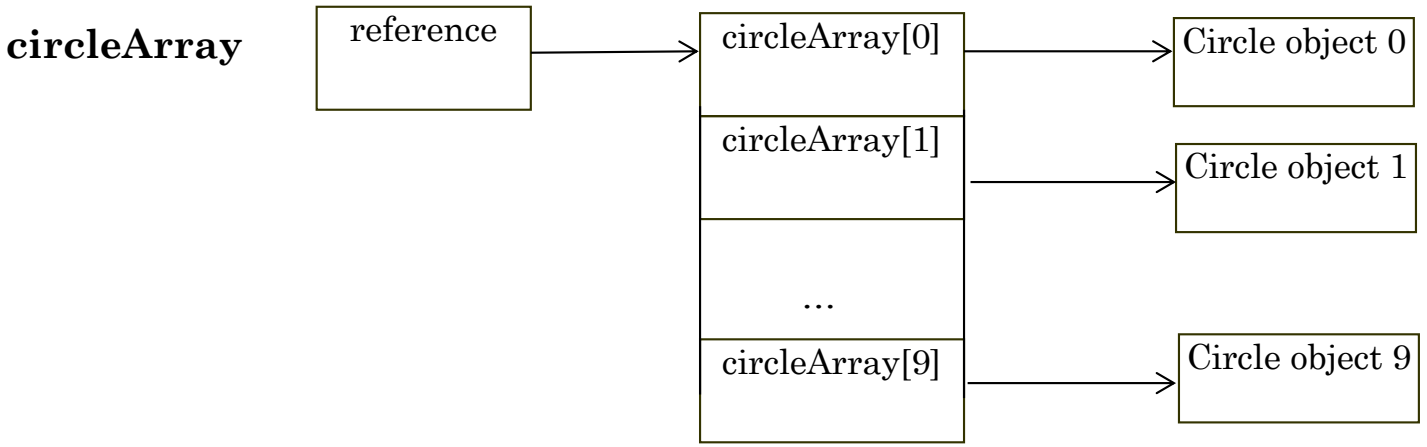
Passing Objects to Methods



Arrays of Objects

```
Circle[] circleArray = new Circle[10];
```

- An array of objects is an *array of reference variables*
- Invoking `circleArray[1].getArea()` involves two levels of referencing
  - circleArray references to the entire array
  - circleArray[1] references to a Circle object



Hexagon

Immutable Objects and Classes

- If the contents of an object **cannot be changed once the object is created**
  => object is called an *immutable object* and its class is called an *immutable class*
- If you delete the set methods, a class **may** become immutable as the attributes are private & **cannot be changed** without a set method
- A class with all private data fields and without mutators is not necessarily immutable

```
public class Student {
  private int id;
  private BirthDate birthDate;

  public Student(int ssn,
      int year, int month, int day) {
    id = ssn;
    birthDate = new BirthDate(year, month, day);
  }

  public int getId() {
    return id;
  }

  public BirthDate getBirthDate() {
    return birthDate;
  }
}
```

```
public class BirthDate {
  private int year;
  private int month;
  private int day;

  public BirthDate(int newYear,
      int newMonth, int newDay) {
    year = newYear;
    month = newMonth;
    day = newDay;
  }

  public void setYear(int newYear) {
    year = newYear;
  }
}
```

```
public class Test {
  public static void main(String[] args) {
    Student student = new Student(111223333, 1970, 5, 3);
    BirthDate date = student.getBirthDate();
    date.setYear(2010); // Now the student birth year is changed!
  }
}
```

Why Immutable?

- For a class to be immutable, it must **mark all data fields private** and **provide no mutator methods** and **no accessor methods** that return a reference to a mutable data field object
- Maximum reliance on immutable objects is widely accepted as a *sound strategy* for creating simple, reliable code (mostly for synchronous access)
- Thread-safe by default (concurrent write never occurs)
- Cacheable

Examples of Immutable classes

- java.lang.String
- Wrapper classes for the primitive types:
  - java.lang.Integer, java.lang.Byte, java.lang.Character, java.lang.Short, java.lang.Boolean, java.lang.Long, java.lang.Double, java.lang.Float
- java.lang.StackTraceElement (used in building exception stacktraces)
- Enum classes
- java.math.BigInteger and java.math.BigDecimal,
- java.io.File
- java.awt.Font, java.awt.BasicStroke, java.awt.Color,
- java.util.Locale
- java.util.UUID, java.net.URL and java.net.URI
- java.net.Inet4Address, java.net.Inet6Address, Java.net.InetSocketAddress

Scope of Variables

- The scope of instance and static variables is the entire class
- Variables can be declared anywhere inside a class
- The scope of a local variable starts from its declaration and continues to the end of the block that contains the variable
- A local variable must be initialized explicitly before it can be used

this as Keyword

- this = name of a reference that refers to the object itself
- this => reference a class's hidden data fields
- this => enables a constructor to invoke another constructor of the same class

Reference the Hidden Data Fields

```
public class Foo {
  private int i = 5;
  private static double k = 0;

  void setI(int i) {
   this.i = i;
  }
  static void setK(double k) {
    Foo.k = k;
  }
}
```

Calling Overloaded Constructor

```
public class Circle {
  private double radius;

  public Circle(double radius) {
    this.radius = radius;
  }

  public Circle() {
    this(1.0);
  }

  public double getArea() {
    return this.radius * this.radius * Math.PI;
  }
}
```

this must be explicitly used to reference the data field radius of the object being constructed

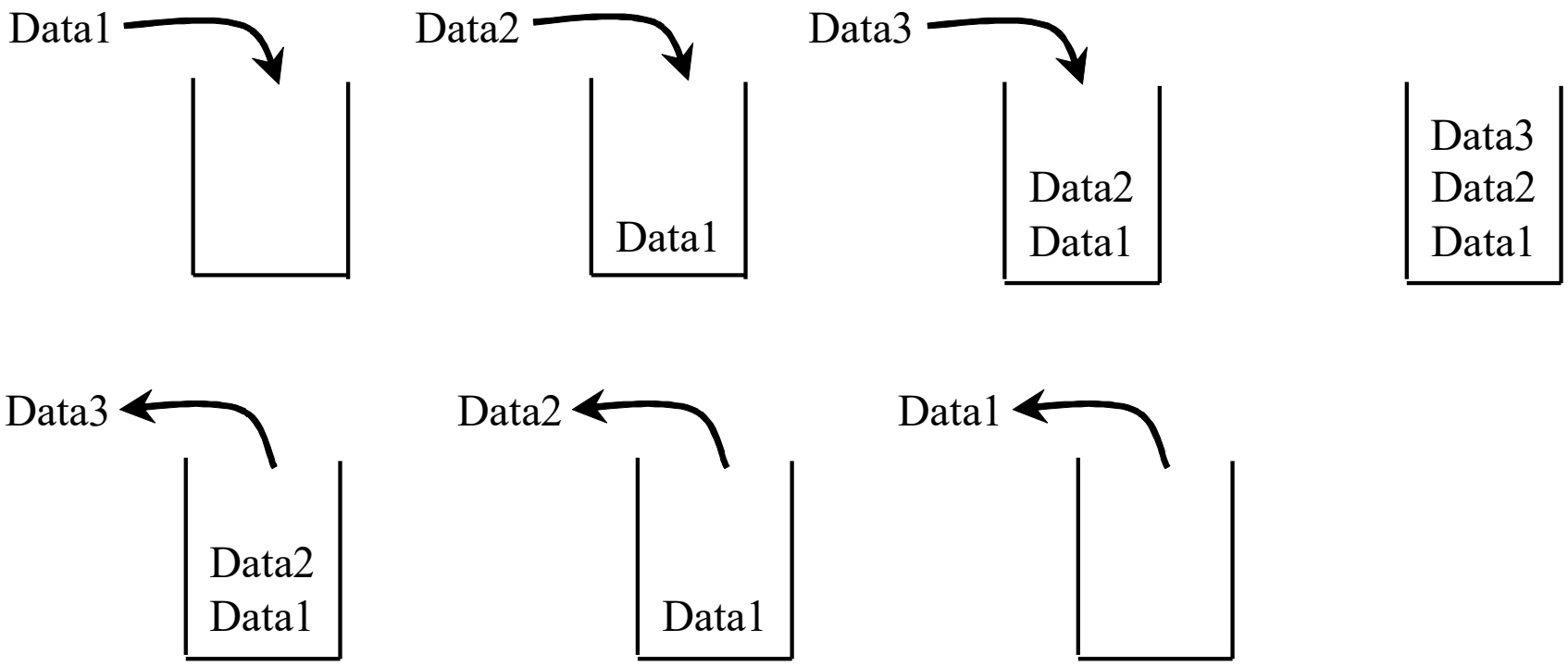
this is used to invoke another constructor

Every instance variable belongs to an instance represented by this, which is normally omitted

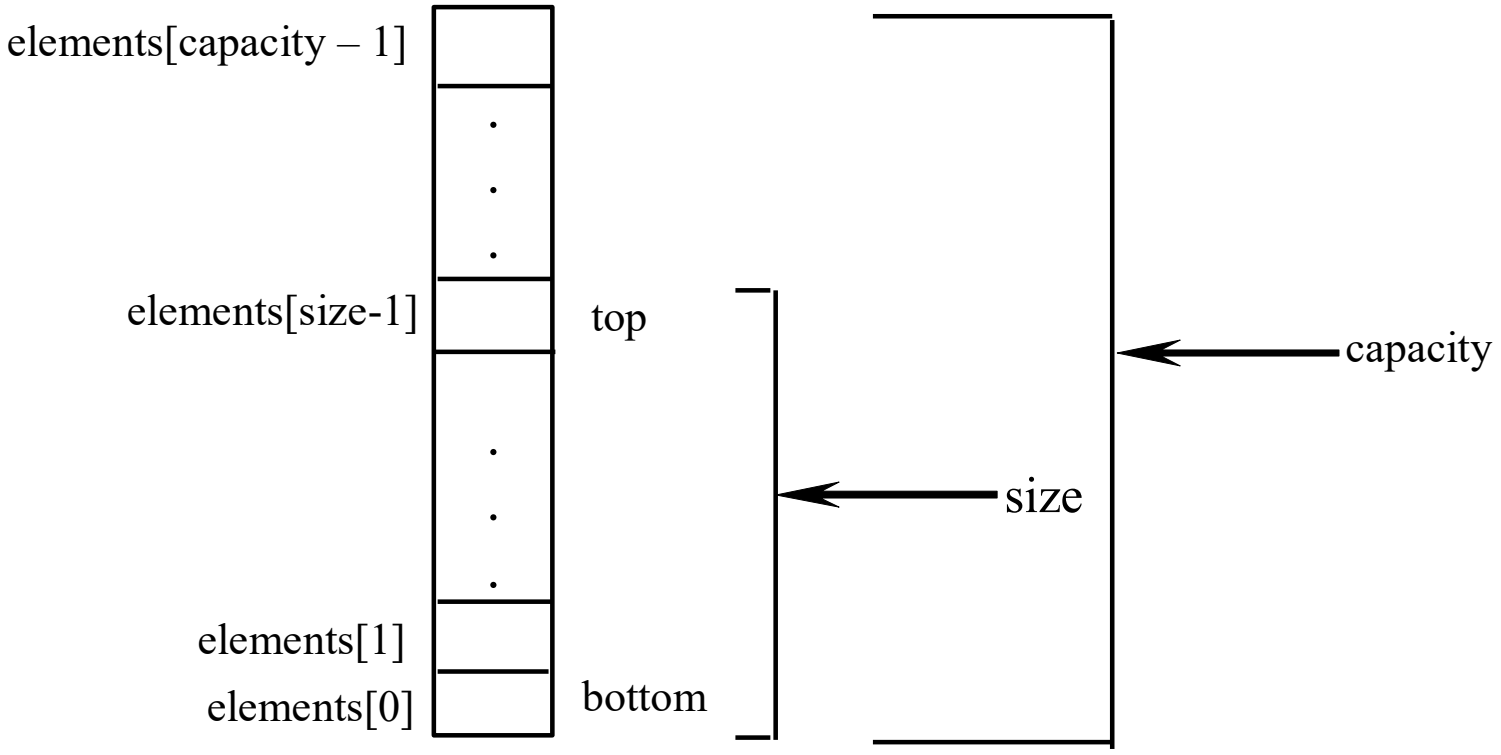
Example: The `StackOfIntegers` Class

StackOfIntegers	
-elements: int[]	An array to store integers in the stack.
-size: int	The number of integers in the stack.
+StackOfIntegers()	Constructs an empty stack with a default capacity of 16.
+StackOfIntegers(capacity: int)	Constructs an empty stack with a specified capacity.
+empty(): boolean	Returns true if the stack is empty.
+peek(): int	Returns the integer at the top of the stack without removing it from the stack.
+push(value: int): int	Stores an integer into the top of the stack.
+pop(): int	Removes the integer at the top of the stack and returns it.
+getSize(): int	Returns the number of elements in the stack.

Designing the StackOfIntegers Class



Implementing `StackOfIntegers` Class



Designing a Class

- (**Coherence**) A class should describe a **single** entity, and all the class operations should logically fit together to support a coherent purpose
  - You can use a class for students, but you should not combine students and staff in the same class, because students and staff have different entities
- (**Separating responsibilities**) A single entity with too many responsibilities can be broken into several classes to separate responsibilities
  - The classes String, StringBuilder, and StringBuffer all deal with strings, but have different responsibilities
    - String deals with immutable strings
    - StringBuilder creates mutable strings
    - StringBuffer is similar to StringBuilder except that StringBuffer contains synchronized methods for updating strings

Designing a Class, cont.

- Classes are designed for reuse
  - Users can incorporate classes in many different combinations, orders, and environments
- Follow standard Java programming style and naming conventions
- Place the data declaration before the constructor
- Place constructors before methods
- Always provide a constructor and initialize variables to avoid programming errors

Designing a Class, cont.

- Provide a public no-arg constructor and override the *equals*() and *toString*() methods defined in the Object class whenever possible
- Therefore, you should design a class that:
  - Imposes no restrictions on what or when the user can do with it
  - Design the properties to ensure that the user can set properties in any order, with any combination of values
  - Design methods to function independently of their order of occurrence